

Module2: Critical Thinking
Option 2: Predicting Future Sales

Christine DeLuna
CSC 580: Dr. Nguyen
August 2, 2026

Building and Evaluating a Neural Network with TensorFlow and Keras

Introduction

Artificial neural networks are one of the foundational technologies behind modern artificial intelligence. Unlike traditional machine learning algorithms that rely on manually engineered rules, neural networks learn complex relationships directly from data through repeated training and optimization. TensorFlow and Keras provide a flexible framework for building, training, evaluating, and deploying these models for a variety of prediction tasks. In this assignment, I developed a neural network capable of predicting the expected revenue of a new video game based on historical sales data and several descriptive features, including critic ratings, genre, pricing, exclusivity, and platform characteristics.

The project followed the complete supervised learning workflow. First, the historical training and testing datasets were normalized using the MinMaxScaler provided by Scikit-learn. Scaling the data to a range between 0 and 1 is an important preprocessing step because neural networks perform more efficiently when all input features exist within a similar numerical range. The scaled datasets were then used to train and evaluate the neural network.

Neural Network Design and Training

The neural network was implemented using TensorFlow with the Keras Sequential API. The model accepted nine input features representing characteristics of each video game and produced a single continuous output representing the predicted total earnings. Three fully connected (Dense) hidden layers were used with the Rectified Linear Unit (ReLU) activation function, while the output layer used a linear activation function since the objective was to predict a continuous numerical value rather than classify data into categories.

Before training, the model was compiled using the Adam optimizer and Mean Squared Error (MSE) as the loss function. The Adam optimizer efficiently updates the model's parameters during training by adjusting the weights based on prediction error, while MSE measures how far the model's predictions differ from the actual earnings. The network was trained for 50 epochs with shuffled training data, allowing the model to repeatedly learn from all available examples while reducing the possibility of learning patterns based solely on the order of the data.

Throughout the training process, the loss steadily decreased with each epoch, indicating that the neural network successfully learned the relationships between the input features and the expected revenue. The decreasing loss demonstrated that the model continually improved its predictions as it adjusted thousands of internal weights through backpropagation and optimization.

Figure 1 **Neural Network Training Progress Across 50 Epochs**

Note. Screenshot of the TensorFlow/Keras model training output showing the reduction in Mean Squared Error (MSE) over 50 training epochs. Figure created by the author using TensorFlow 2.21 and Keras 3.

```

Epoch 1/50
32/32 - 0s - 15ms/step - loss: 0.0205
Epoch 2/50
32/32 - 0s - 757us/step - loss: 0.0029
Epoch 3/50
32/32 - 0s - 720us/step - loss: 0.0011
Epoch 4/50
32/32 - 0s - 611us/step - loss: 5.9029e-04
Epoch 5/50
32/32 - 0s - 626us/step - loss: 3.4843e-04
Epoch 6/50
32/32 - 0s - 653us/step - loss: 2.2915e-04
Epoch 7/50
32/32 - 0s - 659us/step - loss: 1.6367e-04
Epoch 8/50
32/32 - 0s - 784us/step - loss: 1.3405e-04
Epoch 9/50
32/32 - 0s - 835us/step - loss: 1.0532e-04
Epoch 10/50
32/32 - 0s - 703us/step - loss: 9.7593e-05
Epoch 11/50
32/32 - 0s - 806us/step - loss: 8.4108e-05
Epoch 12/50
32/32 - 0s - 814us/step - loss: 6.9113e-05
Epoch 13/50
32/32 - 0s - 783us/step - loss: 6.5221e-05
Epoch 14/50
32/32 - 0s - 877us/step - loss: 5.3546e-05
Epoch 15/50
32/32 - 0s - 1ms/step - loss: 4.8404e-05
Epoch 16/50
32/32 - 0s - 949us/step - loss: 4.0620e-05
Epoch 17/50
32/32 - 0s - 707us/step - loss: 3.0517e-05

```

Figure 2
Data Preprocessing and Feature Scaling Using MinMaxScaler

Note. Screenshot of the Python implementation used to normalize the training and testing datasets using Scikit-learn's `MinMaxScaler`. The output confirms the scaling parameters applied to the `total_earnings` feature and the successful creation of the scaled datasets.

```

main.py  assignments/.../README.md  README.md  .gitignore  scale_data.py
1  from sklearn.preprocessing import MinMaxScaler
2
3  # -----
4  # Load the original datasets
5  # -----
6  training_data_df = pd.read_csv("sales_data_training.csv")
7  test_data_df = pd.read_csv("sales_data_test.csv")
8
9  # -----
10 # Scale all columns between 0 and 1
11 # -----
12 scaler = MinMaxScaler(feature_range=(0, 1))
13
14 scaled_training = scaler.fit_transform(training_data_df)
15

```

Run scale_data

```

/Users/christine/.pyenv/versions/3.11.4/bin/python3 /Users/christine/PycharmProjects/CSC580/assignments/CTA2-NeuralNetwork/scale_data.py
Note: total_earnings values were scaled by multiplying by 0.0000036969 and adding -0.115913
Scaled data files created successfully!
Process finished with exit code 0

```

Model Evaluation and Prediction

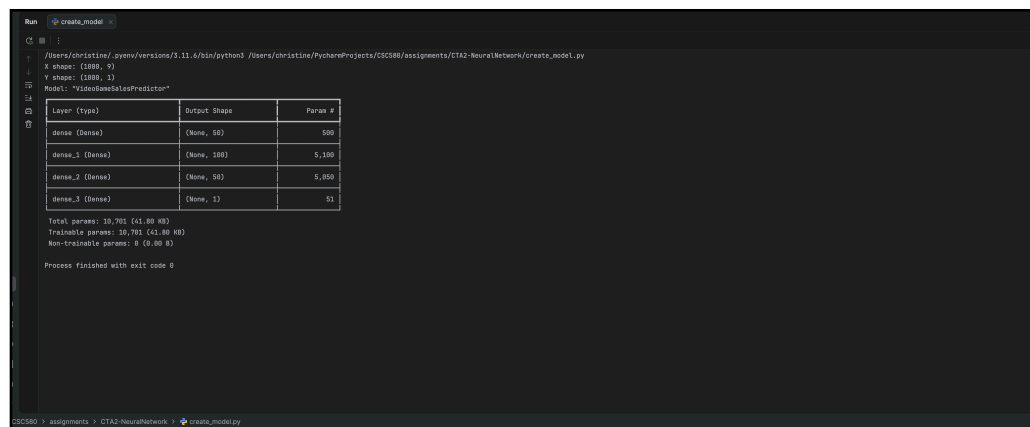
After training, the model was evaluated using a separate testing dataset that had not been used during training. This independent evaluation is critical because it measures the model's ability to generalize to previously unseen data rather than simply memorizing the training examples. The resulting Mean Squared Error (MSE) was approximately 9.88×10^{-5} , indicating a very small prediction error on the normalized testing data. Because the testing error remained low and comparable to the final training loss, the results suggest that the neural network generalized well without significant evidence of overfitting.

The trained model was then saved to disk and later reloaded to predict the earnings of a hypothetical new video game. After converting the normalized prediction back to its original dollar value using the scaling parameters generated during preprocessing, the model predicted approximately \$200,216.31 in total earnings. This final step demonstrated how a trained neural network can be deployed to make predictions for new data without requiring additional training.

During implementation, I updated portions of the provided starter code to reflect current TensorFlow and Keras best practices. Specifically, I replaced the deprecated `input_dim` argument with an explicit `Input(shape=(9,))` layer, which is the recommended approach in modern versions of Keras. Additionally, although Keras now recommends saving models using the native `.keras` format, I retained the legacy `.h5` format to remain compatible with the assignment requirements and the provided prediction workflow.

Figure 3 Neural Network Architecture Used for Revenue Prediction

Note. Model summary generated by TensorFlow/Keras illustrating the architecture of the neural network developed for predicting video game sales revenue. The model accepts nine input features and consists of three fully connected (Dense) hidden layers containing 50, 100, and 50 neurons, followed by a single linear output neuron that predicts total earnings. The network contains 10,701 trainable parameters and no non-trainable parameters.



```
Run create_model
/Users/christine/gymn/versions/3.11.6/bin/python3 /Users/christine/PycharmProjects/CS338H/assignments/CTA2-NeuralNetwork/create_model.py
K shape: (1000, 9)
V shape: (1000, 1)
Model: "VideoGameSalesPredictor"
Layer (type) Output Shape Param #
-----
dense (Dense) (None, 50) 500
dense_1 (Dense) (None, 100) 5,100
dense_2 (Dense) (None, 50) 5,050
dense_3 (Dense) (None, 1) 51
Total params: 10,701 (41.80 KB)
Trainable params: 10,701 (41.80 KB)
Non-trainable params: 0 (0.00 B)
Process finished with exit code 0
```

Conclusion

This assignment provided practical experience with the complete lifecycle of supervised deep learning, including data preprocessing, neural network design, model training, evaluation, persistence, and prediction. More importantly, it demonstrated how neural networks learn by iteratively minimizing prediction error rather than relying on manually programmed rules. Although this project focused on predicting video game revenue, the same supervised learning workflow can be applied to many real-world prediction problems where complex relationships exist within structured data.

As a founder focused on improving care for individuals with autoimmune diseases and other complex chronic conditions, I found this assignment particularly meaningful. Rather than predicting financial outcomes, a similar neural network architecture could be trained to recognize patterns that precede disease flares by learning from longitudinal patient data such as laboratory results, symptoms, wearable device metrics, medications, sleep quality, and other clinical indicators. While such a model would require significantly larger and carefully validated medical datasets, this assignment reinforced how deep learning could eventually contribute to earlier intervention, improved disease monitoring, and more personalized care. Understanding the complete machine learning workflow—from preprocessing and training to evaluation and deployment—provides a strong technical foundation for pursuing these types of healthcare applications responsibly.

References

Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, Ł., Kudlur, M., ... Zheng, X. (2016). *TensorFlow: Large-scale machine learning on heterogeneous systems*. <https://www.tensorflow.org/>

Chollet, F. (2021). *Deep learning with Python* (2nd ed.). Manning Publications.

Geitgey, A. (n.d.). *Building deep learning: Keras Tree Master 03*. GitHub.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.